

PIBIC

Henrique Poledna

O que é extração de funcionalidades?

Imagem crua = milhões de pixels.

O que importa: bordas, texturas, formas, padrões

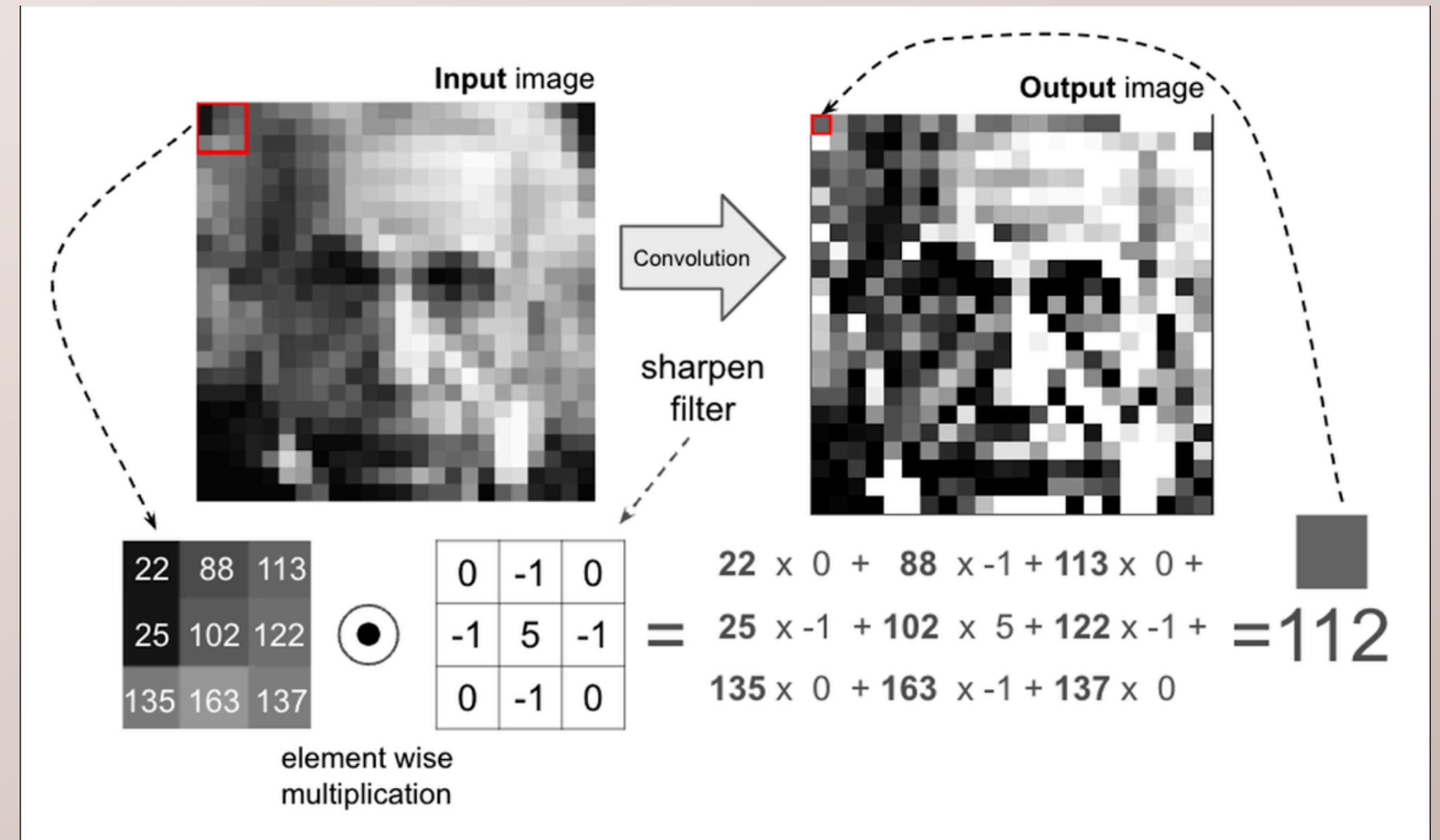
Feature extraction tenta fazer uma compressão de informação e redução de ruído

Dados brutos:

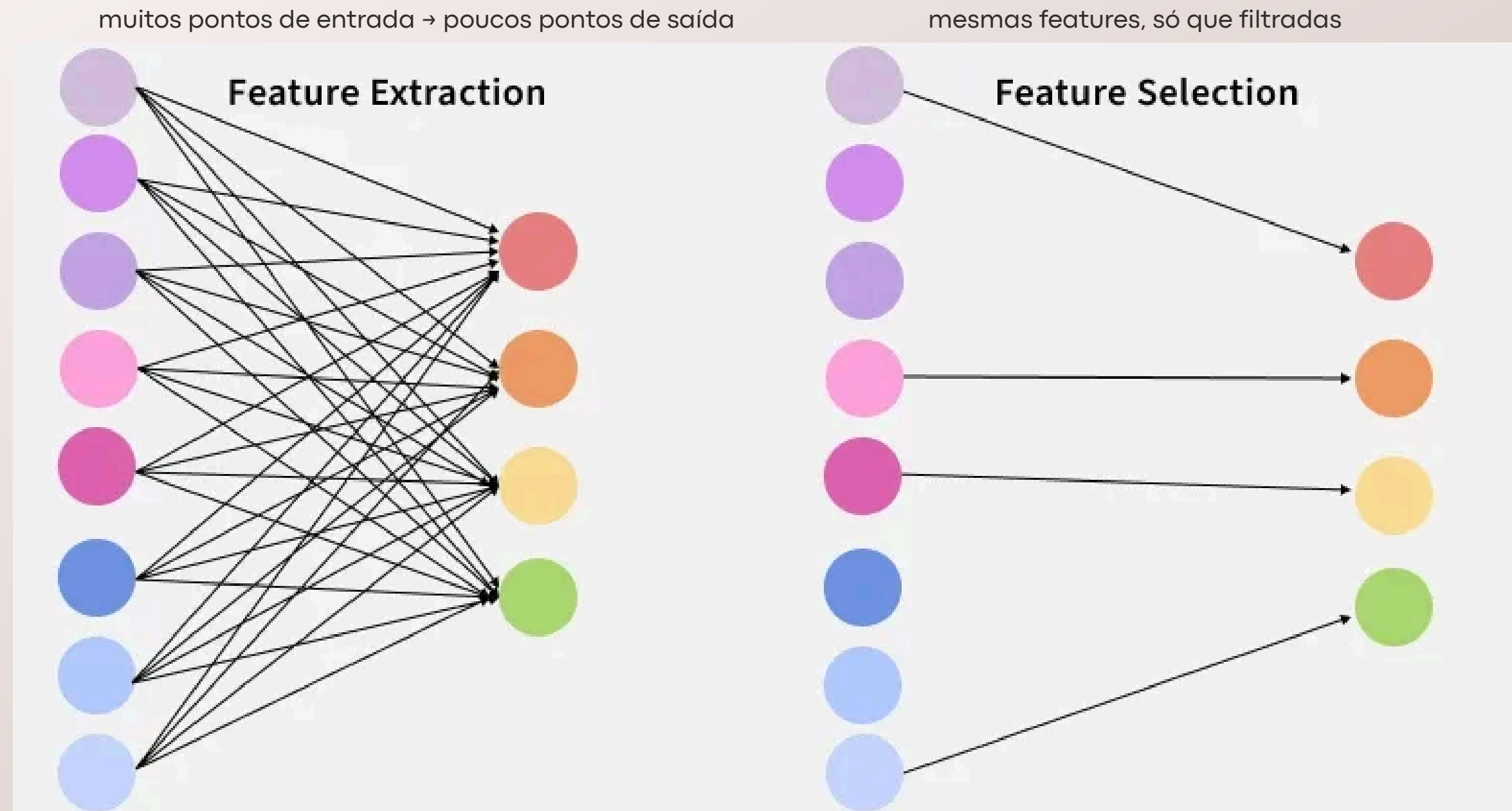
temperatura
batimentos
pressão

Features que importam:

febre alta?
taquicardia?
inflamação?



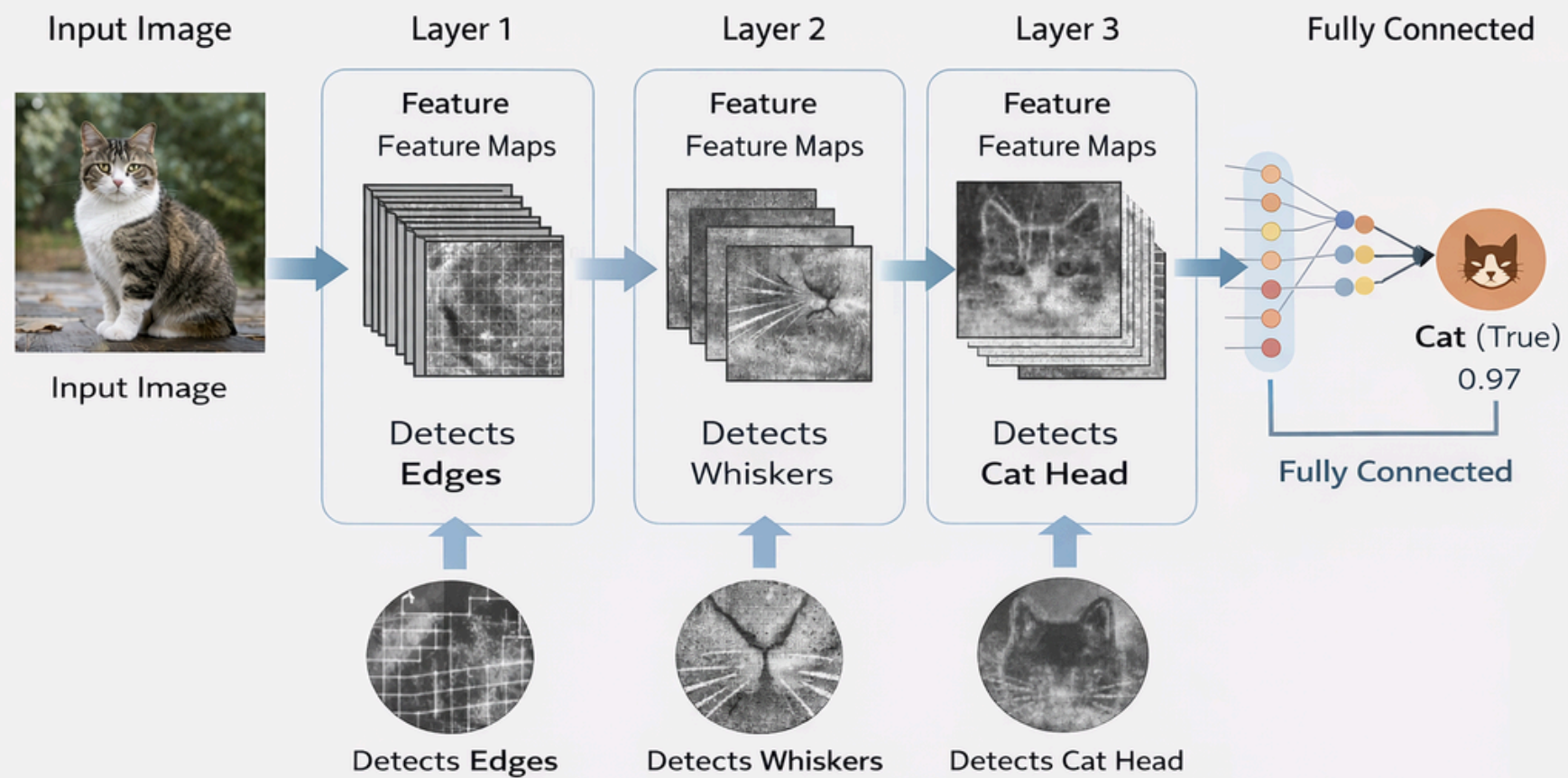
O que é extração de funcionalidades?



Idade, altura, peso

Feature Selection:
Usar apenas idade e peso (remover altura).
Feature Extraction:
Criar nova feature: $IMC = \text{peso} / \text{altura}^2$.

Convoluções



Low-level features

São convoluções normais (kernels 3x3, 5x5 etc).
Elas aprendem coisas muito simples:

- bordas (edges)
- linhas horizontais/verticais
- cantos

Mid-level features

Agora elas recebem como entrada os mapas de bordas.

Então passam a aprender combinações de bordas:

- texturas
- padrões repetidos
- partes de objetos

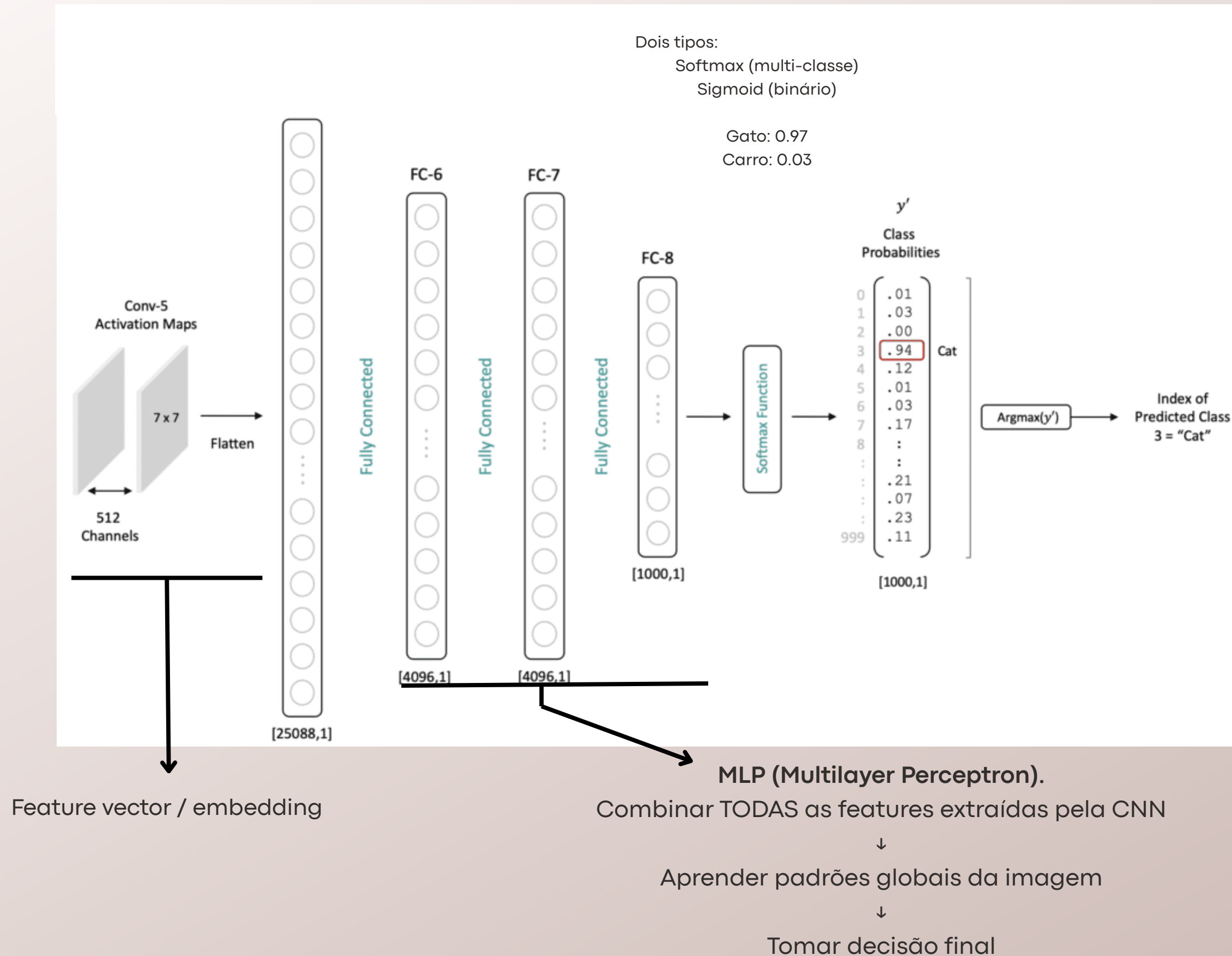
High-level features

Convoluções mas agora recebem partes de objetos como entrada.

Então aprendem combinar partes - objeto inteiro:

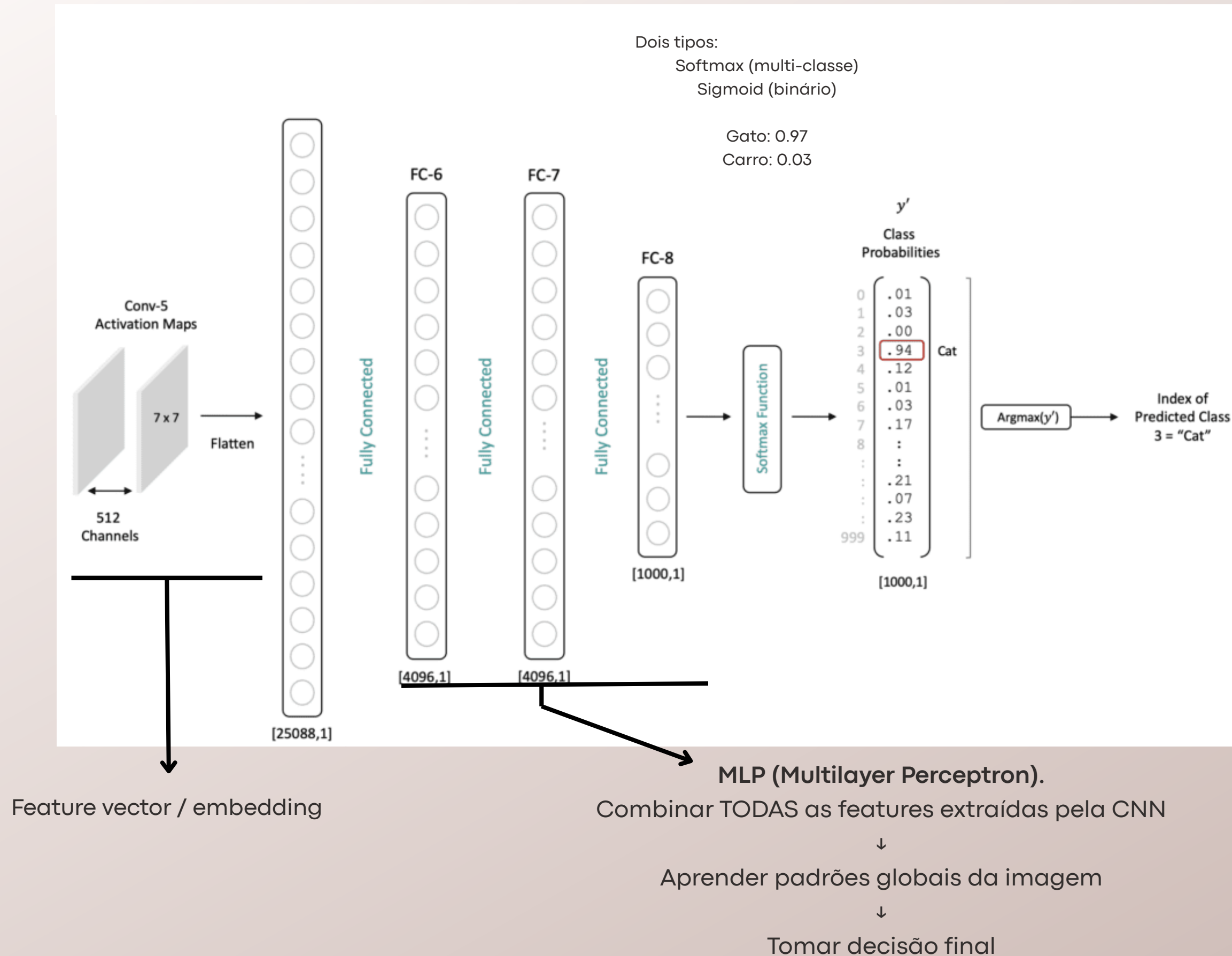
- rosto de gato
- corpo do gato
- silhueta completa
- postura do animal

Fully Connected layer (FC) || Dense layer



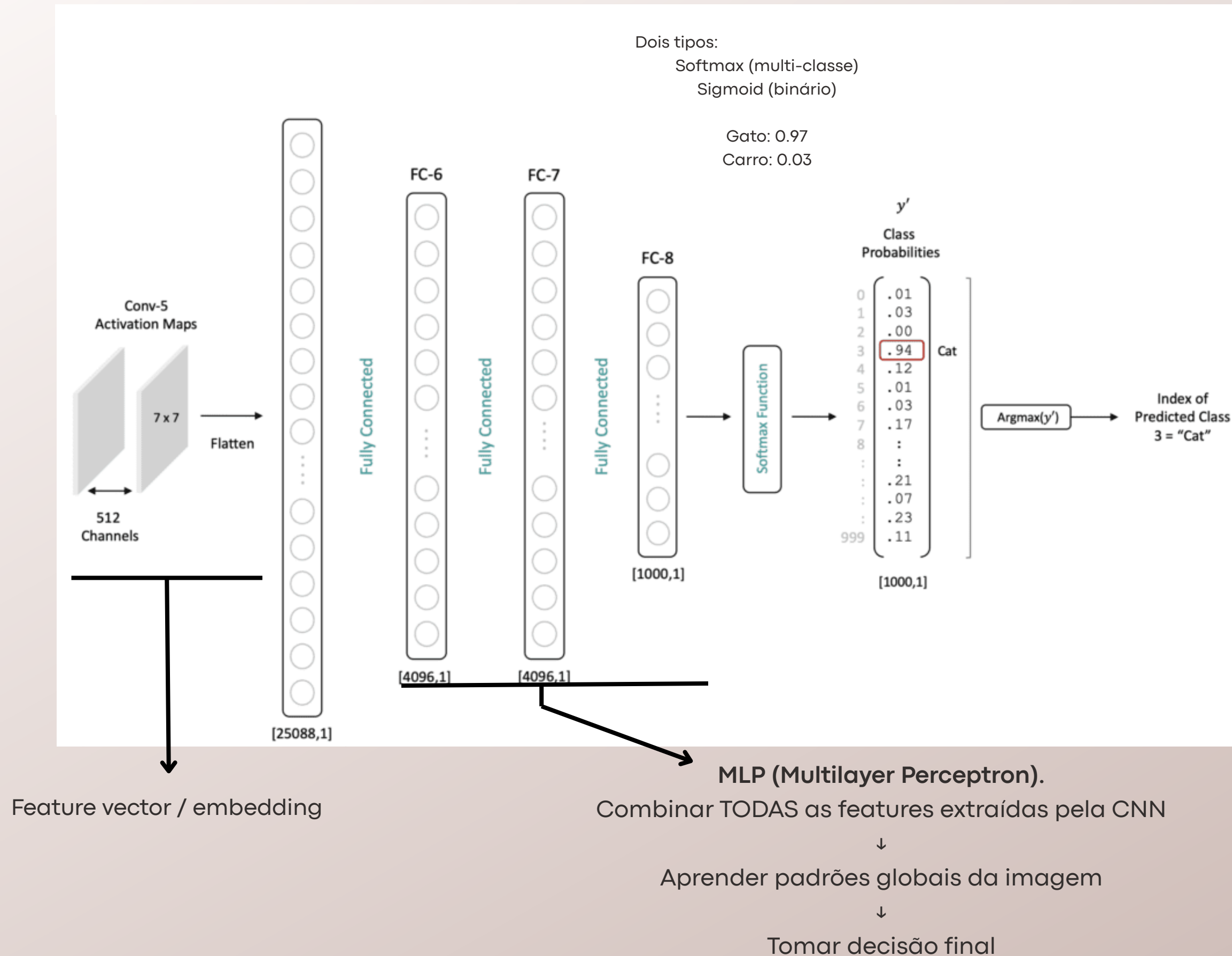
FC6 — Global Feature Integration
25088 → 4096 (~102M parâmetros)
Transforma features para conceitos globais.
Aprende coisas como:
textura de pelo
formato da cabeça
Função: comprimir e integrar todas as features

Fully Connected layer (FC) || Dense layer



FC7 — Semantic Reasoning
4096 → 4096 neurônios
Combina os conceitos aprendidos na FC6.
Aqui começa o raciocínio semântico.
Exemplos:
rosto + orelhas pontudas + pelo → felino
rodas + janelas → carro
asas + bico → pássaro
Função: combinar conceitos de alto nível.

Fully Connected layer (FC) || Dense layer



FC8 — Classifier / Decision Layer
4096 → N classes
Última camada da rede.
Transforma os conceitos em probabilidades de classe usando Softmax.
Exemplo:
gato → 0.93
cachorro → 0.06
carro → 0.01
Função: tomar a decisão final da rede.

Fully Connected layer (FC) || Dense layer

Antes era:

Linear(512 → 1000 classes ImageNet)

Agora virou:

Linear(512 → 2 classes spoof/bonafide)

```
82 model.fc = nn.Sequential(  
83     nn.Dropout(0.3), Durante o treino ele desliga neurônios aleatoriamente  
84     nn.Linear(model.fc.in_features, 2)  
85 )  
86
```

A saída da ResNet antes da FC é X e usa como base, sendo 512, ent usa 512

Entrada	Saída
vetor de 512 features	2 números

Transfer Learning

FEATURE EXTRACTOR FIXO (CONGELAR TUDO)

A CNN já sabe extrair features boas.

Só precisamos de um classificador novo, treinando apenas o FC

```
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
for param in model.parameters():
    param.requires_grad = False # congela CNN inteira

model.fc = nn.Linear(model.fc.in_features, 2)

optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
```

Transfer Learning

FINE-TUNING COMPLETO (DESCONGELAR TUDO)

O que treina?

Tudo:

Conv layers + FC

```
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(model.fc.in_features, 2)
```

A rede começa com conhecimento prévio, mas continua aprendendo.

Exemplos:

- espectrogramas
- imagens médicas
- satélite
- raio-X

Etapa	Métrica	Valor
Validação (Dev)	EER (Menor Erro)	0.85%



Transfer Learning

TREINAR SÓ AS ÚLTIMAS CONVOLUÇÕES (BEST PRACTICE)

O que treina?

Layer-1(última stage convolucional) + FC

Features de baixo nível do ImageNet não transferem bem para espectrogramas.

Etapa	Métrica	Valor
Validação (Dev)	EER (Menor Erro)	0.85%



Etapa	Métrica	Valor
Validação (Dev)	EER (Menor Erro)	3.92%

Camadas iniciais aprendem padrões universais → manter
Camadas finais aprendem semântica → adaptar
Então treinamos apenas as camadas profundas.

```
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# congela tudo
for param in model.parameters():
    param.requires_grad = False

# descongela última stage convolucional
for param in model.layer4.parameters():
    param.requires_grad = True

model.fc = nn.Linear(model.fc.in_features, 2)
```

Save Score para TDCF

Arquivo	Score	Predição
PA_D_0000001	6.03e-08	bonafide
PA_D_0000011	0.9620	spoof
PA_D_0000021	0.0238	bonafide
PA_D_0000022	0.7326	spoof
PA_D_0000026	1.22e-13	bonafide
PA_D_0000028	0.9994	spoof

Arquivo	Ataque	Label	Score
PA_E_0000001	-	bonafide	0.99958789467928
PA_E_0000002	BB	spoof	0.0
PA_E_0000007	-	bonafide	0.9999999999791664
PA_E_0000015	AC	spoof	5.960464477539062e-07
PA_E_0000024	AB	spoof	8.106231689564147e-06
PA_E_0000028	-	bonafide	0.999942675960483

```
score_file_dev = f"scores/{args.model_name}_DEV_scores.txt"
score_file_eval = f"scores/{args.model_name}_EVAL_scores.txt"
```

```
logits = model(x) → [z_bonafide , z_spoof] → [ -2.1 , 3.4 ]
probs = torch.softmax(logits, dim=1) → [p_bonafide , p_spoof] → [0.041 , 0.959]
```

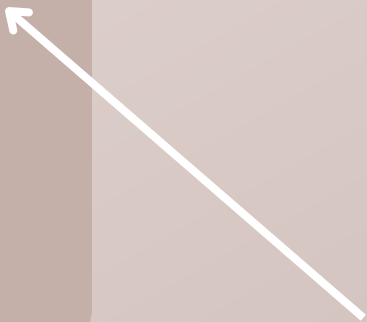
Score	Interpretação
< 0.001	bonafide fortíssimo
0.001 – 0.1	bonafide
0.1 – 0.5	tendência bonafide
0.5 – 0.9	tendência spoof
> 0.9	spoof fortíssimo

Save Score para TDCF

coluna	significado
speaker	id do locutor
utt	id do áudio
env	ambiente de gravação
attack	tipo de ataque físico
label	bonafide / spoof

Código	Ideia
AA	replay ruim
AB	replay médio
AC	replay bom
BA	outro setup
BB	outro setup
CC	replay super realista

PA_0016 PA_E_00000001 cbc - bonafide
PA_0029 PA_E_00000002 bab BB spoof
PA_0044 PA_E_00000003 bac BA spoof
PA_0019 PA_E_00000004 cab CC spoof
PA_0021 PA_E_00000005 baa AB spoof
PA_0029 PA_E_00000006 bac BB spoof



PA_cm_protocols/ASVspoof2019.PA.cm.eval.trl.txt

:)